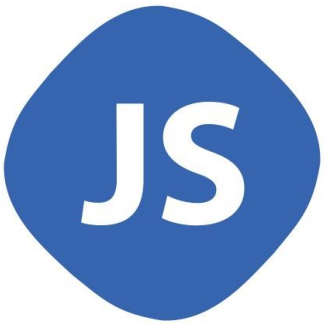


Всплывающие окна

- Введение
- Всплывающие окна
- Передача данных в всплывающее окно
- Передача данных из всплывающего окна в основное
- Всплывающее окно и форма
- Всплывающее окно и backend
- Закрытие всплывающего окна

Введение

Всплывающее окно (англ. *pop-up*) — окно, открываемое на экране компьютера в результате выполнения какой-либо операции. Всплывающие окна используются для реализации **пользовательского интерфейса** в веб-приложениях.

Всплывающие окна имеют широкий диапазон применений — от полезного для потребителя программного обеспечения до злоумышленного.

Наиболее широкое применение всплывающие окна приобрели для размещения рекламных сообщений в сети Интернет. Содержащие рекламу всплывающие окна открываются на **переднем** (*pop-up*) или на **заднем** фоне (англ. *pop-under*) (что еще хуже).

Pup-under является более изощрённым приёмом распространения нежелательных для пользователя рекламных сообщений, поскольку не привлекает внимание пользователя до тех пор, пока активное окно не будет закрыто, и затрудняет возможность определения источника появления всплывающего окна.

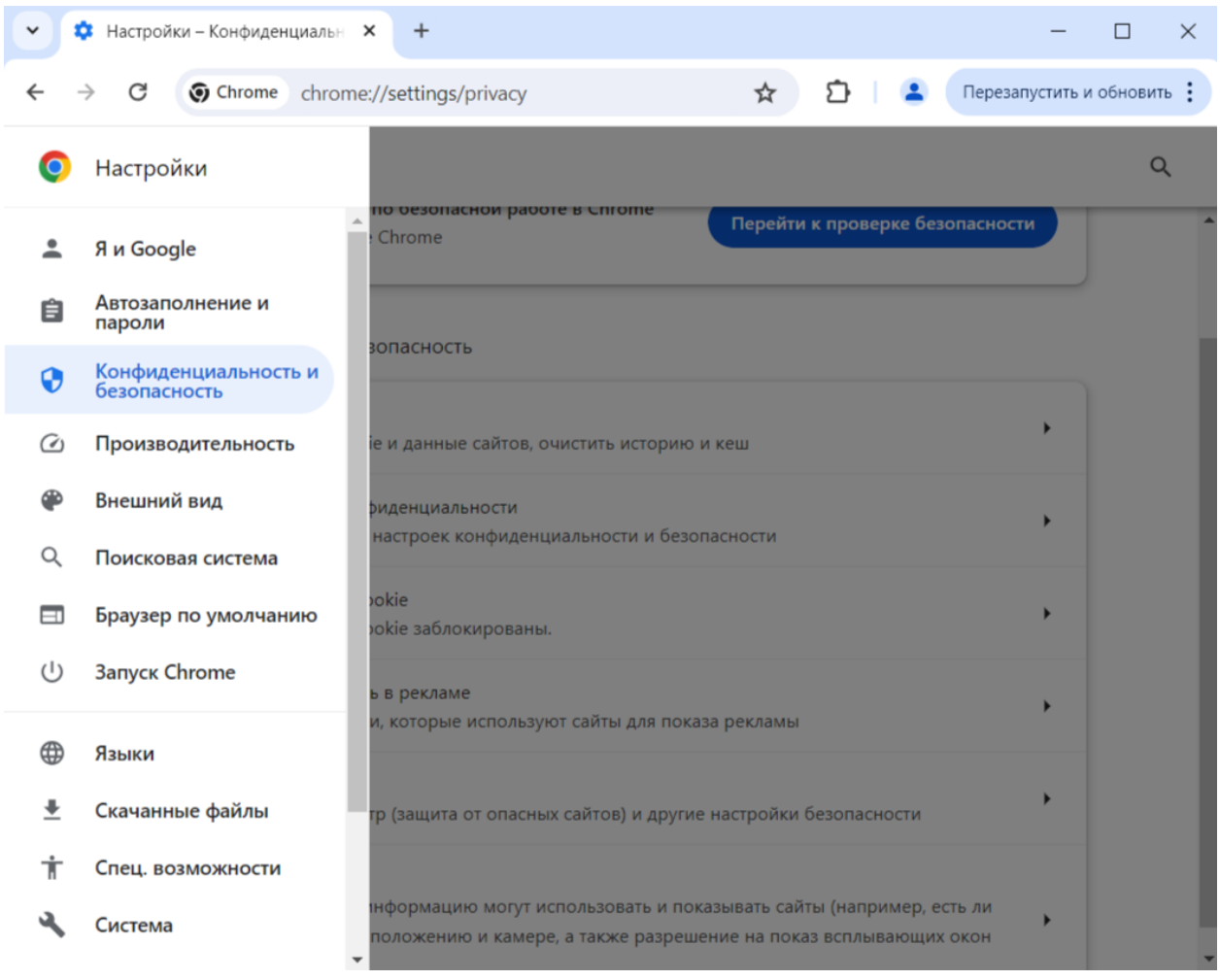
Массовое применение всплывающих окон для размещения рекламы в сети Интернет привело к появлению в веб-браузерах функции **блокирования всплывающих окон**. Практически каждый современный браузер включает

такую функцию, а также даёт возможность пользователю создавать реестр сайтов, для которых показ всплывающих окон разрешён или запрещён.

> ***Важно***. Браузер может блокировать создание всплывающих окон. Для тестирования кода урока найдите настройки вашего браузера, отвечающие за использование всплывающих окон.

Лично мне работа с всплывающими окнами браузера **Google Chrome** кажется наиболее адекватной и предсказуемой (но это не обязательно так ;)

Настройку браузера можно выполнить на вкладке **Конфиденциальность и безопасность.**

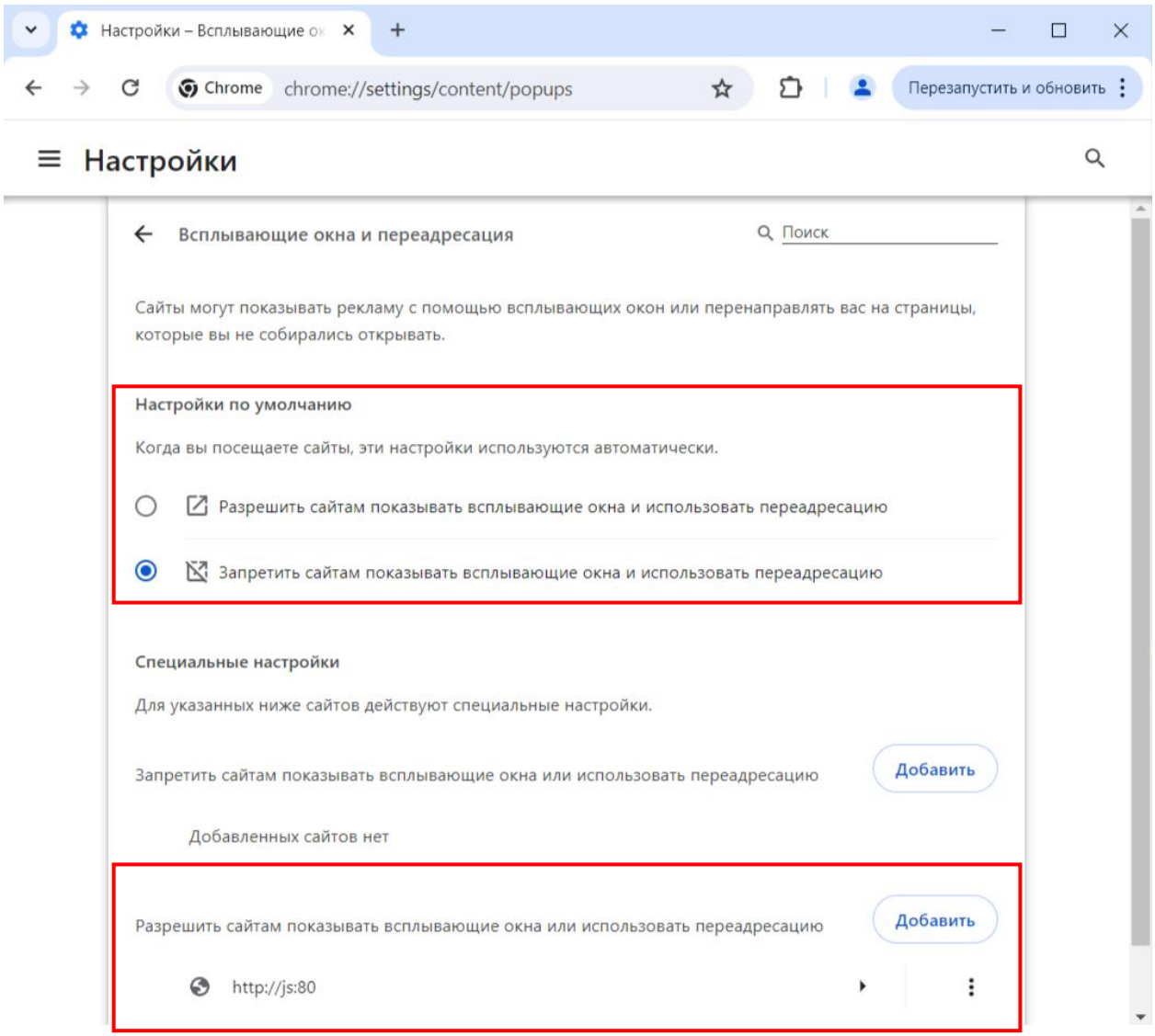


Полный путь к настройке всплывающих окон:

1. Конфиденциальность и безопасность

2. Настройки сайта
3. Всплывающие окна и переадресация.

Вы может полностью **отключить** / **включить** все всплывающие окна. Или произвести выборочную настройку для каждого сайта.



По завершении урока верните настройки к исходному состоянию.

Всплывающие окна

Для их создания традиционно используется технология JavaScript, хотя возможны и другие способы реализации. Язык JavaScript позволят программно управлять окнами веб-браузера.

Для этого объект window предоставляет ряд методов. Так, метод **open**() открывает определенный ресурс в новом окне или вкладке браузера.

Синтаксис метода open

```
window.open(URL, name, params);
```

Где:

- **URL – необязательный** параметр. Указывает URL страницы, которая будет открыта в окне. Если данный параметр отсутствует **окно будет открыто пустым**.
- **name** – **необязательный** параметр. Указывает **способ открытия** или **имя окна**. Возможные значения:
  - **name** – устанавливает имя окна;
  - **_blank** – URL загружается в новом окне;
  - **_parent** – URL загружается в родительском фрейме;
  - **_self** – URL заменяет текущую страницу;
  - **_top** – URL заменяет все фреймы, которые могут быть загружены.
- **params** – **необязательный** параметр. Параметры должны быть отделены запятой, без каких-либо пробелов. Доступны следующие параметры.

  Позиция окна:
  - **width** (число) – ширина окна в пикселах (не менее 100);
  - **height** (число) – высота окна в пикселах (не менее 100);
  - **left** (число) – расстояние от левого края экрана до левой границы окна в пикселах;
  - **top** (число) – расстояние от верхнего края экрана до верхней границы окна в пикселах;

  Панели окна:
  - **location** (yes|no) – наличие у окна поля адреса;
  - **menubar** (yes|no) – наличие у окна панели меню;
  - **resizable** (yes|no) – сможет ли пользователь менять размер окна;
  - **scrollbars** (yes|no) – наличие у окна полос прокрутки;

- **status** (yes|no) – наличие у окна поля статуса;
- **toolbar** (yes|no) – наличие у окна панели инструментов.

Популярные события окна

При работе с всплывающими окнами вам придется использовать некоторые, наиболее популярнные события окна:

- **onresize**() – изменение размера окна;
- **onscroll**() – прокрутка окна;
- **onload**() – страница загружена;
- **onfocus**() – фокус установлен;
- **onblur**() – фокус потерян.

Стоит учитывать, что манипуляции с всплывающими окнами лучше выполнять по **действию пользователя**, например, по нажатию на кнопку, в ином случае браузеры могут заблокировать подобные окна.

Первые примеры выполним с минимальным набором параметров, рассмотрим возможные настройки всплывающих окон.

Создаем пустое всплывающее окно

Демонстрационный пример **example_1** представляет минимально возможный набор настроек всплывающего окна.

example_1

```
<script>
    // создаем всплывающее окно
    window.open(
        '',
        'NewWindow'
    )
```

```
</script>
```

Начнем применять параметры окна.

example_2

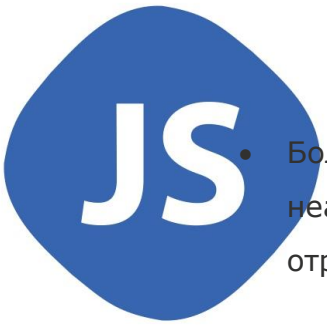
```
<script>
    // создаем всплывающее окно
    window.open(
        '',
        'NewWindow',
        'left=0,top=0,width=500,height=500'
    )
</script>
```

Параметры можно хранить в отдельной переменной (или нескольких переменных).

example_3

```
<script>
    let params = 'left=500,top=0,width=-
    500,height=400,toolbar=no,scrollbars=no,resizable=no';
    // создаем всплывающее окно
    window.open(
        '',
        'NewWindow',
        params
    )
</script>
```

Обратите внимание:



- Большинство современных браузеров **самостоятельно исправит** неадекватные значения параметров, например, нулевые width/height или отрицательные left/top.

- Если аргумент параметров (params) при вызове метода **open**() отсутствует или пустой, будут использованы **настройки окна по умолчанию**.
- Если строка параметров передана, но некоторые параметры **yes**/**no** пропущены, то считается, что указано значение **no**, так что соответствующие возможности будут отключены (если на это нет ограничений со стороны браузера).
- Если координаты **left**/**top** не заданы, браузер попытается открыть новое окно **рядом** с предыдущим открытым окном.

- Если не заданы размеры окна **width**/**height**, браузер откроет новое окно с теми же размерами, что и предыдущее открытое окно.

Загружаем в всплывающее окно внешний файл

Пустое окно неплохой вариант, но, скорее всего, загружать **подготовленный шаблон** всплывающего окна придется чаще.

example_4. **example_4.html**

```
<script>
    // создаем всплывающее окно
    // подгружаем в окно файл target.html
    window.open(
        'target.html',
        'NewWindow',
        'left=500,top=200,width=600,height=400,toolbar=yes,scrollbars=yes,resizable=yes'
    )
```

```
</script>
```

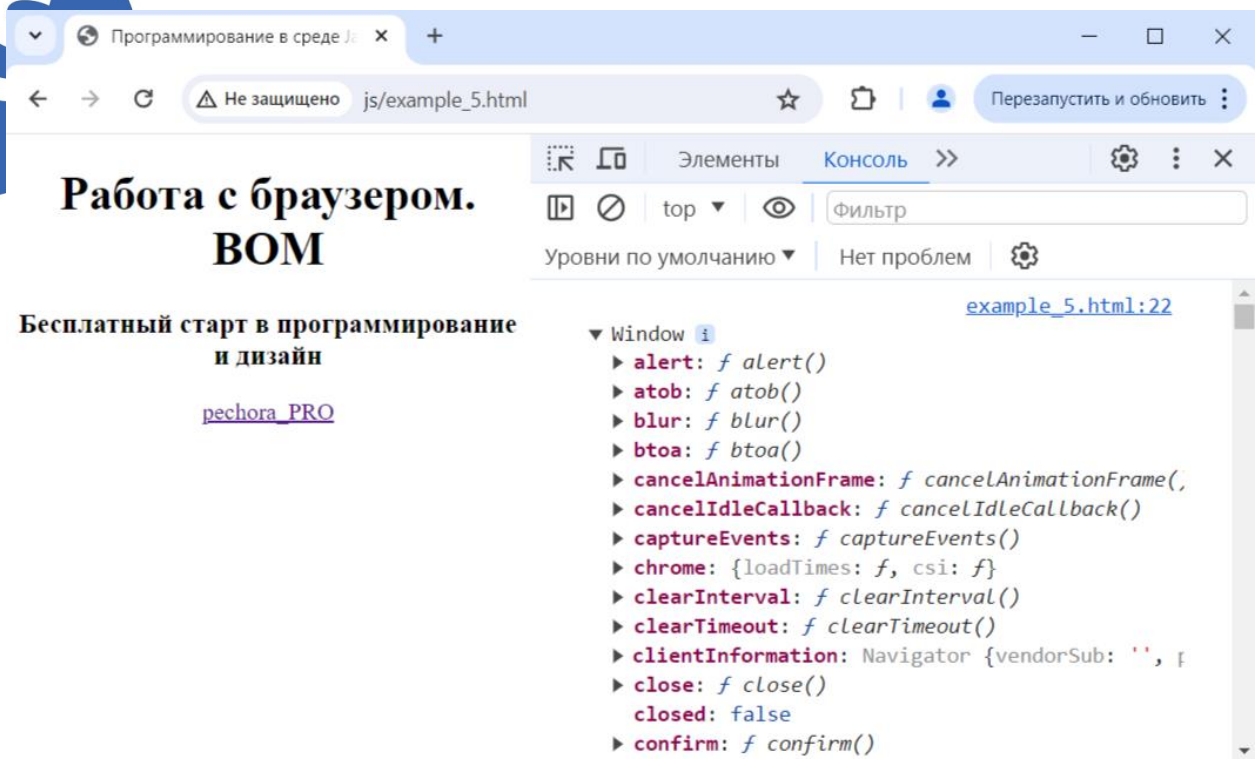
Целевой файл **target.html** одинаковый для примеров **example_4**, **example_5** и представлен ниже.

Вызов метода **open**() возвращает **ссылку на новое окно**. Эта ссылка может быть использована для управления **свойствами** окна (например, изменения положения) и его **содержимым**.

example_5. **example_5.html**

```
<script>
    // создаем всплывающее окно
    // подгружаем в окно файл target.html
    // сохраняем ссылку на созданное окно
    let mywin = window.open(
        "target.html",
        "_blank",
        "left=50,top=20,width=900,height=500,toolbar=yes,scrollbars=yes,resizable=yes"
    );
    // ссылка на всплывающее окно
    console.log(mywin);
</script>
```

Ссылка на всплывающее окно выводит в консоль **стандартный объект window**.



Целевой файл – **target.html**.

target.html

```
<h2>Здравствуйте! Я target.html</h2>
```

Это введение в всплывающие окна. Дальше будет интереснее и немного сложнее.

Передача данных в всплывающее окно

При работе с окнами может быть организован обмен данными в двух направлениях:

- из **основного** к всплывающему;
- от **всплывающего** к основному.

Из основного окна можно обратиться к всплывающему (ровно как и наоборот). Передавать данные можно **двумя способами**:

- Во время создания окна.

- После загрузки окна.

Передача данных во время создания окна

В демонстрационном примере **example_6** рассмотрим самый простой способ обмена информацией между окнами, запись данных с использованием метода **document.write**().

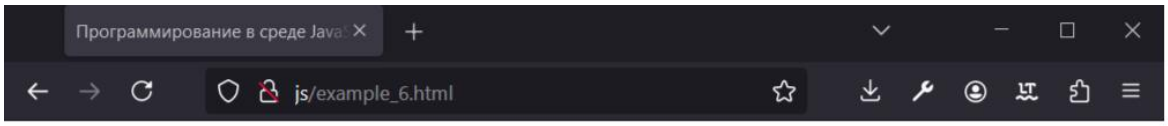
example_6. **example_6.html**

```
<script>
    // создаем всплывающее окно
    // подгружаем в окно файл target.html
    // сохраняем ссылку на созданное окно
    let mywin = window.open(
        "target.html",
        "_blank",
        "left=50,top=20,width=700,height=300,toolbar=yes,scrollbars=yes,resizable=yes"
    );
    // используя ссылку на окно записываем в документ динамический контент
    mywin.document.write('<h1>Это динамический контент</h1>');
    mywin.document.write('<div style="margin:10px; padding:10px; border:1px solid #c0c0c0"><h2>');
    mywin.document.write('Образование создало огромное множество людей, которые умеют читать, но не способны определить, что заслуживает чтения.');
    mywin.document.write('<br>(<i>Джордж Маколей Тревелян</i>)</h2></div>');
</script>
```

Целевой файл **target.html**. Обратите внимание, метод **write**() приведет к удалению **имеющегося** контента файла, загружаемого в всплывающее окно.

example_6. target,html

```
<header align="center">
    <h1>Работа с браузером. BOM</h1>
</header>
<h2>Контент файла будет удален</h2>
<footer align="center">
    <h3>Бесплатный старт в программирование и дизайн</h3>
    <a href="https://vk.com/pechora_pro"
    target="_blank">pechora_PRO</a>
</footer>
```



Работа с браузером. BOM

Бесплатный старт в программирование и дизайн

pechora_PRO



Это динамический контент

Образование создало огромное множество людей, которые умеют читать, но не способны определить, что заслуживает чтения.
(*Джордж Маколей Тревелян*)



Передача данных после загрузки окна

Второй способ записи данных подразумевает использование события **onload**() всплывающего окна.

Усложним код и будем передавать в **всплывающее** окно **динамический контент**, полученный из внешнего файла-скрипта.

example_7. **example_7.html**

```
<!-- подключаем внешние данные -->
<script src="course.js"></script>
<!-- повесим обработчик на событие oncliick -->
<a href="#" onclick="openWin()">Открыть список курсов в новом окне</a></p>
<script>
    // функция-обработчик
    function openWin () {
        event.preventDefault();
        out = "<ol>";
        // представляем массив курсов в виде списка
        course.forEach( (item) => {
            out += `<li><h3>${item.topic}</h3></li>`
        });
        out += "</ol>";
        // загружаем в всплывающее окно файл target.html
        let mywin =
        window.open("target.html","_blank","width=600,height=700"
        );
        // в документ окна загружаем новый контент
```

```
        mywin.onload = function () {
            mywin.document.getElementById("target").innerHTML =
            out;
        }
    }
</script>
```

Целевой файл содержит **блок контейнер** для вставки контента.

example_7. target.html

```
<!-- блок-контейнер -->
<div id="target"></div>
```

В демонстрационном примере **example_8** формируемый из массива динамический контент используется как в **основном**, так и в **всплывающем** окне.

example_8. example_8.html

```
<script>
    let out = "";
    // формируем контент для текущего документа
    guitars.forEach( (item) => {
        out += `
            <h4><a id="${item.id}" href="#"
            onclick="openWin(event)">${item.name}</h4>
            <img src="images/${item.image}" width=100px>
        `
    });
    // вставка контента в текущий документ
```

```
    document.getElementById("main").innerHTML = out;
    // контент формируется при клике по ссылке
    function openWin (e) {
        guitar = guitars.find( (item) => {
            return e.target.id == item.id;
        });
        // формируем контент для вывода
        let out = `
            <h2>${guitar.name}</h2>
            <p><b>${guitar.preview}</b></p>
            <div>
                ${guitar.description}
            </div>
            <img src="images/${guitar.image}" width="300px">
        `
        // загружаем в всплывающее окно файл target.html
        let mywin =
        window.open("target.html","_blank","width=600,height=700"
        );
        // событие load всплывающего окна
        mywin.onload = function () {
            // вставка контента в всплывающее окно
            mywin.document.getElementById("target").innerHTML =
            out;
        }
    }
</script>
```

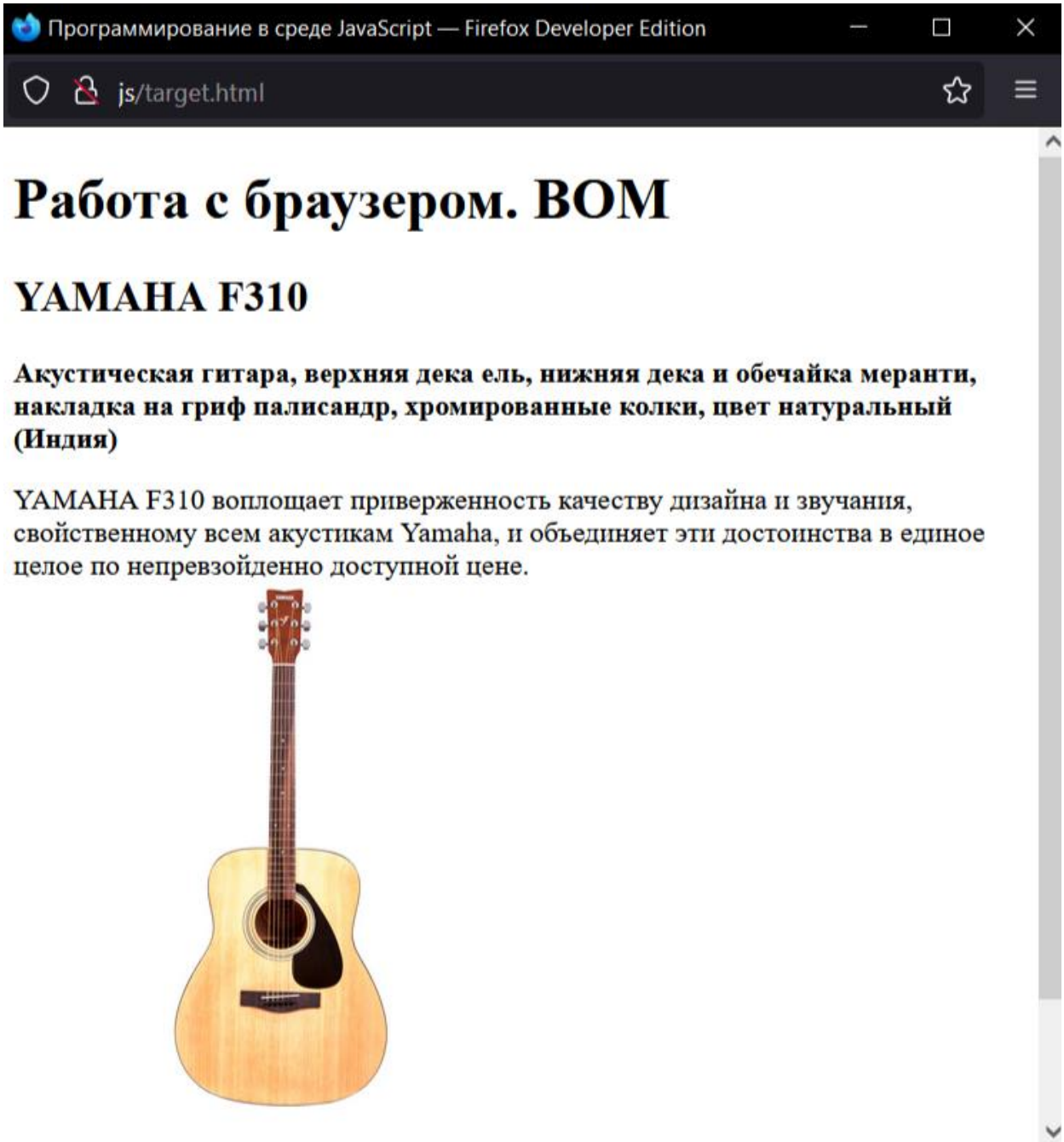
Целевой файл содержит **блок контейнер** для вставки контента.

example_8. **target.html**

```
<!-- блок-контейнер -->

<div id="target"></div>
```

Всплывающее окно демонстрационного примера представлено на следующем скриншоте.





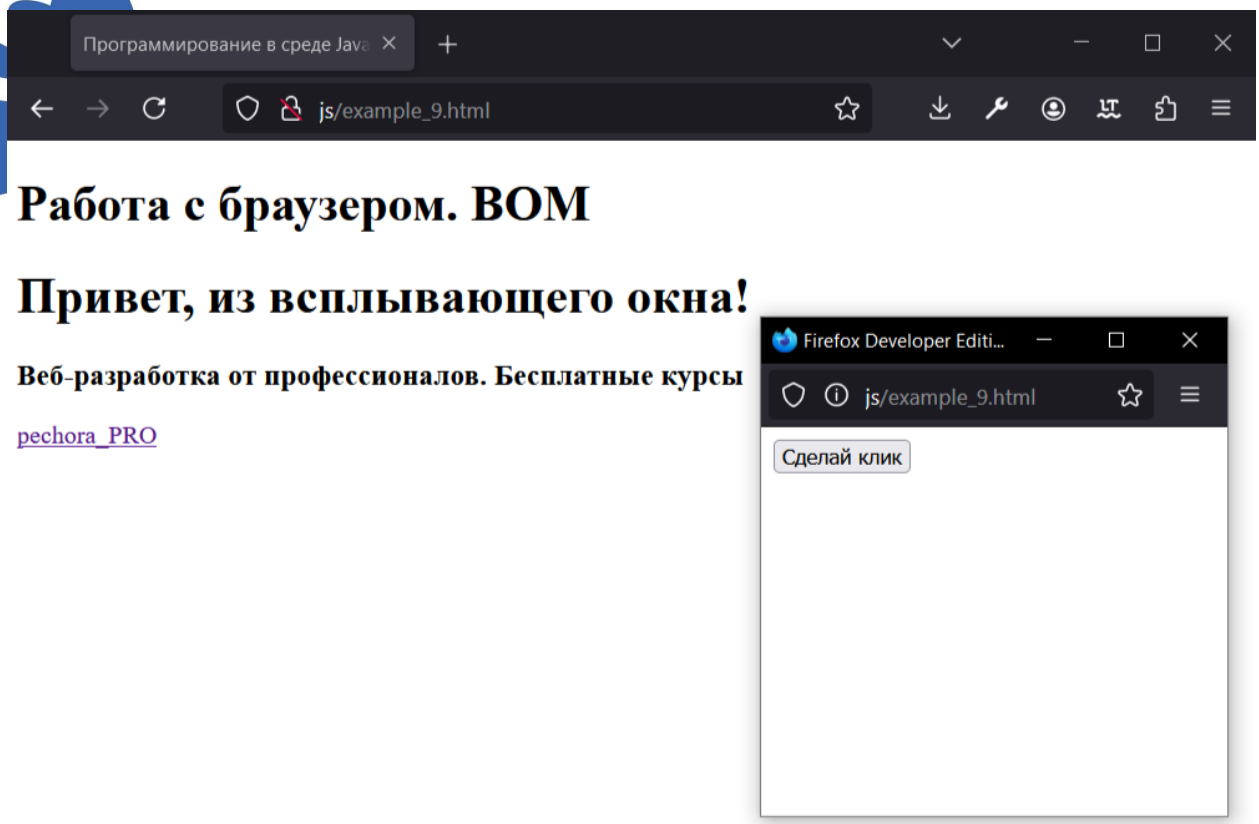
Передача данных из всплывающего окна в основное

Рассмотрим обратное направление передачи данных, из **всплывающего** окна в **основное**.

example_9. **example_9.html**

```
<!-- блок для вставки контента -->
<div id="main"></div>
<script>
    // сохраняем ссылку на всплывающее окно
    let mywin =
    window.open('target.html','popUp','left=400,top=200,width=300
    ,height=250');
    // в всплывающее окно записываем кнопку
    mywin.document.write('<button id="btn">Сделай
    клик</button>');
    // получаем ссылку на кнопку всплывающего окна
    let btn = mywin.document.querySelector('#btn');
    // навешиваем на кнопку событие click
    btn.addEventListener('click', () => {
        // из ссылки на всплывающее окно получаем доступ к
        родительскому
        mywin.opener.document.getElementById("main").innerHTML =
        '<h1>Привет, из всплывающего окна!</h1>';
    })
</script>
```

Целевой файл **target.html** – пустой файл со стандартной HTML-разметкой.



example_10. example_10.html

```
<!-- блок для вставки контента -->
<div id="main"></div>
<button onclick="openWin()">Открыть список</button>
<script>
    // функция запуска событий страницы
    function openWin () {
        let out = "";
        // формируем контент для всплывающего окна
        citate.forEach( (val, key) => {
            str =  val.substring(0,25);
            // формируем строку для вывода
            out += `
```

```
            <h3><a class="citate" data-num="${key}"
            href="#">${str} ...</h3>
        `
});
// сохраняем ссылку на всплывающее окно
var mywin =
window.open("target.html","pupUp","width=600,height=700")
;
// создаем событие загрузки всплывающего окна
mywin.onload = function () {
    // загружаем контент в всплывающее окно
    mywin.document.getElementById("target").innerHTML =
    out;
    // получаем коллекцию ссылок всплывающего окна
    links = mywin.document.querySelectorAll("a.citate");
    // перебираем коллекцию, навешиваем на каждую ссылку
    событие click
    for(let link of links) {
        link.addEventListener("click", function() {
            // выполняем поиск цитаты, по которой
            кликнули
            out = citate.find( (val, key) => {
                return this.dataset.num == key;
            });
            // вставка цитаты в родительское окно
            mywin.opener.document.getElementById("main").
            innerHTML = "<h2>" + out + "</h2>";
        })
```

```
        }
    }
}
</script>
```

example_10. **target.html**

```
<!-- блок-контейнер -->
<div id="target"></div>
```

Обратите внимание на логику работы демонстрационных примеров:

example_8:

- **основное** окно - список гитар;
- **всплывающее** окно – информация о выбранной гитаре

example_11:

- **всплывающее** окно - список гитар;
- **основное** окно – информация о выбранной гитаре

example_11. **example_11.html**

```
<script>
    // функция запуска событий страницы
    function openWin () {
        let out = "";
        // формируем контент для всплывающего окна
        guitars.forEach( (item) => {
            out += `
                <h4><a class="guit" id="${item.id}"
                href="#">${item.name}</h4>
```

```
            <img src="images/${item.image}" width=100px>
        `
});
// сохраняем ссылку на всплывающее окно
var mywin =
window.open("target.html","pupUp","width=600,height=700")
;
// создаем событие загрузки всплывающего окна
mywin.onload = function () {
    // загружаем контент в всплывающее окно
    mywin.document.getElementById("target").innerHTML =
    out;
    // получаем коллекцию ссылок всплывающего окна
    links = mywin.document.querySelectorAll("a.guit");
    // перебираем коллекцию, навешиваем на каждую ссылку
    событие click
    for(let link of links) {
        link.addEventListener("click", function() {
            let out = "";
            // выполняем поиск гитары, по которой
            кликнули
            guitar = guitars.find( (item) => {
                return this.id == item.id;
            });
            // формируем контент
            out = `<h2>${guitar.name}</h2>
            <p><b>${guitar.preview}</b></p>
```

```
                    <div>${guitar.description}</div>
                    <img src="images/${guitar.image}"
                    width="300px">
                    `;
                    // вставляем в родительское окно новый
                    контент
                    mywin.opener.document.getElementById("main").
                    innerHTML = out;
                })
            }
        }
    }
</script>
```

example_11. **target.html**

```
<!-- блок-контейнер -->
<div id="target"></div>
```

Каждый раз управляя динамическим содержимым окон, мы писали JS-код в основном файле. Но никто нам не мешает **писать код и в всплывающем окне**.

В демонстрационном примере передадим в всплывающее окно строку запроса с **GET-параметрами**.

example_12. **example_12.html**

```
<a href="#" onclick="openWin()">Открыть новое окно</a></p>
<script>
    // функция запуска событий страницы
```

```
    function openWin () {
        event.preventDefault();
        // сохраняем ссылку на всплывающее окно
        let mywin =
        window.open("target.html?par1=Veni&par2=vidi&par3=vici","
        pupUp","width=600,height=700");
        // создаем событие загрузки всплывающего окна
        mywin.onload = function () {
            // добавляем контент в всплывающее окно
            mywin.document.getElementById("target").innerHTML =
            '<h3>Однажды Гай Юлий Цезарь сказал:</h3>';
        }
    }
</script>
```

Объект URL рассмотрим в одном из уроков текущей темы, но сейчас поработать с этим объектом будет уместно.

example_12. **target.html**

```
<!-- блок-контейнер -->
<div id="target"></div>
<script>
    // создаем объект url из адресной строки
    let url = new URL(window.location.href);
    // разбираем объект на параметры
    let par1 = url.searchParams.get('par1');
    let par2 = url.searchParams.get('par2');
    let par3 = url.searchParams.get('par3');
```

```
    // формируем строку
    out = `<h1>${par1} ${par2} ${par3}!</h1>`;
    document.write(out);
</script>
```

Всплывающее окно и форма

Форма – одна из основных технологий передачи данных между сценариями. А поэтому в примере **example_13** тестируем форму.

Обратите внимание, HTML-разметка формы имеет следующие **атрибуты**:

- **hidden** – форма скрыта;
- **target** – данные формы будут вставлены в **адресную строку** всплывающего окна.

example_13. **example_13.html**

```
<div id="main"></div>
<a href="#" onclick="openWin()">Открыть новое окно</a></p>
<script>
    // функция запуска событий страницы
    function openWin () {
        event.preventDefault();
        // сохраняем ссылку на всплывающее окно
        let mywin = window.open("target.html", 'pupUp'
        ,"width=600,height=700");
        // создаем событие загрузки всплывающего окна
        mywin.onload = function () {
            // HTML-разметка формы
```

```
            // обратите внимание на атрибуты:
            // hidden - форма скрыта
            // target - данные формы будут вставлены в адресную
            строку всплывающего окна
            let form = '
                <form hidden action='target.html' target='pupUp'
                id='frm' method='get'>
                <input type="text" name='citate' value="Ум - это
                все. Вы становитесь тем, о чем думаете">
                <input type="text" name='author' value="Будда">
                </form>
            '
            // вставка в документ HTML-разметки формы
            document.getElementById("main").innerHTML = form;
            // mywin.focus();
            // отправка формы
            document.getElementById("frm").submit();
        }
    }
</script>
```

И еще раз поработаем с **GET-параметрами строки запроса**. Но на этот раз параметры сформированы формой.

example_13. **target.html**

```
<script>
    // создаем объект url из адресной строки
    let url = new URL(window.location.href);
```

```
    // разбираем объект на параметры
    let citate = url.searchParams.get('citate');
    let author = url.searchParams.get('author');
    // формируем строку
    out = `<h1>${citate} (${author}).</h1>`;
    document.write(out);
</script>
```

Всплывающее окно и backend

И еще немного накрутим кода. Невозможно обойти вопросы создания всплывающего окна без динамически формируемого на стороне сервера контента (в том числе с использованием баз данных).

> ***Примечание***. Демонстрационные примеры текущего раздела содержат файлы с расширением **.php** – сценарии на языке программирования PHP.
>
> **PHP-сценарии требуют наличия серверного программного обеспечения**.

В примере **example_14** и основное и всплывающее окно имеют инструкции серверных сценариев. При этом обратите внимание, HTML-разметка формы имеет следующие **атрибуты**:

- **hidden** – форма скрыта;
- **action** – обработчк формы – файл с расширением .php;
- **method** – post;
- **target** – данные формы будут вставлены в **тело запроса** всплывающего окна.

example_14. example_14.html

```
<!-- блок для вставки формы -->
<div id="main"></div>
<a href="#" onclick="openWin()">Открыть новое окно</a></p>
<script>
    // функция запуска событий страницы
    function openWin () {
        event.preventDefault();
        // сохраняем ссылку на всплывающее окно
        let mywin = window.open("target.php", 'pupUp'
        ,"width=600");
        // HTML-разметка формы
        // обратите внимание на атрибуты:
        // hidden - форма скрыта
        // action - файл с расширением .php
        // method - post
        // target - данные формы будут вставлены в тело запроса
        всплывающего окна
        let form = `
            <form hidden action='target.php' id='frm'
            target='pupUp' method='post'>
            <input type="text" name='firstName' value="Оскар">
            <input type="text" name='secondName' value="Уайльд">
            <input type="text" name='citate' value="У меня
            непритязательный вкус — достаточно самого лучшего.">
            </form>
        `
```

```
        // вставка в документ HTML-разметки формы
        document.getElementById("main").innerHTML = form;
        // mywin.focus();
        // отправка формы
        document.getElementById("frm").submit();
    }
</script>
```

example_14. **target.php**

```
<?php
    // формируем ответ на стороне сервера
    echo <<<HERE
        <h2>{$_POST['citate']}
        ({$_POST['firstName']} {$_POST['secondName']}).
        </h2>
    HERE;
?>
```

В демонстрационном примере **example_15** напишем полноценный запрос к **базе данных** для извлечения **персональных данных педагога**. База данных **db_distance** прилагается к раздаточному материалу расширенной версии курса.

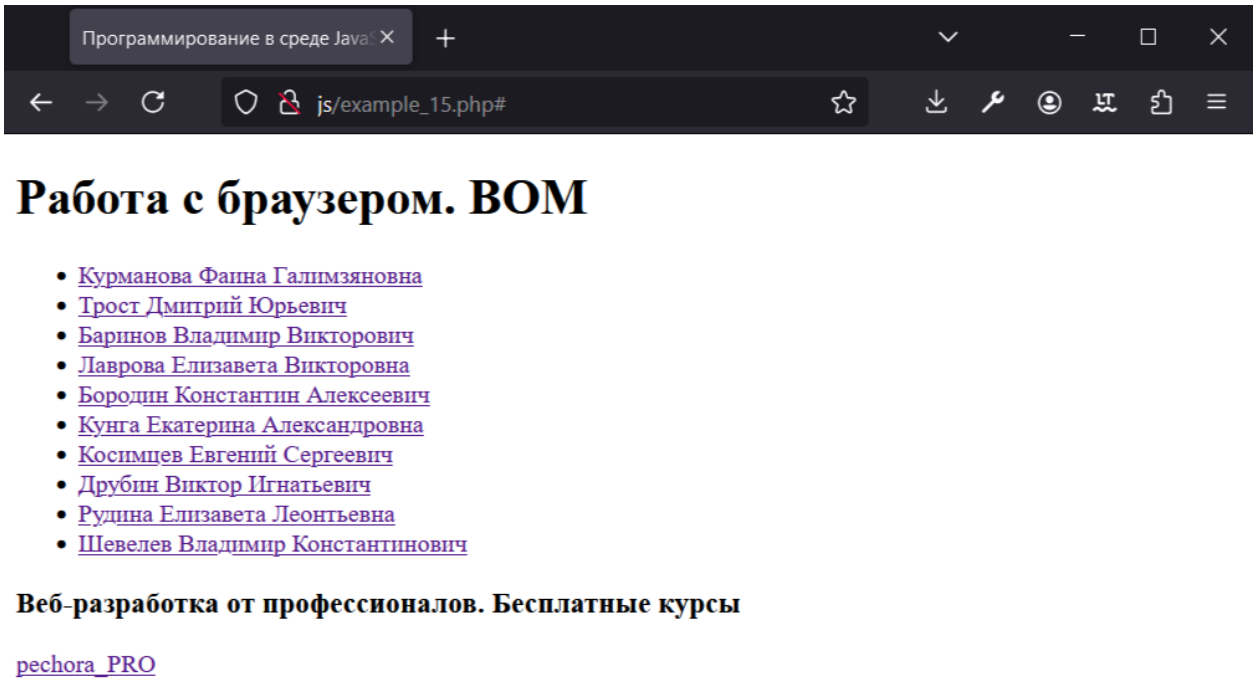
example_15. **example_15.php**

```
<?php
    // подключение к СУБД
    $mysqli = new mysqli('localhost', 'root', '', 'db_distance');
    // строка запроса к базе данных
```

```
    $select = "SELECT * FROM `personnel`";
    $res = $mysqli->query($select);
    $html = '';
    // формирование списка педагогов образовательного заведения
    while ($row = $res->fetch_assoc()) {
        $html .= "
        <li>
        <a href='#' data-id={$row['id_personnel']}
        onclick='fnSend()'>
        {$row['surname']} {$row['name']} {$row['patronymic']}
        </a>
        </li>";
    }
?>
<!-- ... -->
<?php
    // вывод списка
    echo '<ul>' . $html . '</ul>';
?>
<script>
    // функция-обработчик клика по ссылке
    function fnSend() {
        // получаем id ссылки
        let id = event.target.dataset.id;
        // передаем в всплывающее окно идентификатор педагога
        window.open(`target.php?id=${id}`, "pupUp",
        "width=600,height=700");
```

```
}
</script>
```

Внешний вид страницы основного окна представлен на скриншоте.



Долее представлены **код** и **внешний вид** всплывающей страницы.

example_15. target.php

```php
<?php

    // подключение к СУБД

    $mysqli = new mysqli('localhost', 'root', '', 'db_distance');

    // запрос к таблице personnel

    $select_personnel = "SELECT * FROM `personnel` WHERE
    `id_personnel` = " . $_GET['id'];

    $res_personnel = $mysqli->query($select_personnel);

    $row = $res_personnel->fetch_assoc();

    // вывод контента
```

```
echo <<<HERE
    <h2>{$row['surname']} {$row['name']}
    {$row['patronymic']}</h2>
    Должность: <b>{$row['post']}</b><br>
    Категория: <b>{$row['category']}</b><br>
    Уровень образования: <b>{$row['level_edu']}</b><br>
    Рейтинг: <b>{$row['rating']}</b><br>
HERE;
// запрос к таблице education
$select_edu = "SELECT * FROM `education` WHERE `id_personnel`
= " . $_GET['id'];
$res_edu = $mysqli->query($select_edu);
$rows = $res_edu->fetch_all(MYSQLI_ASSOC);
echo '<h3>Образование</h3><hr>';
// циклический вывод контента
foreach ($rows as $row) {
    echo "
        <h3>{$row['institution']}</h3>
        Квалификация: <b>{$row['qualification']}</b><br>
        Специализация: <b>{$row['specialty']}</b>
        ({$row['years']})<br>
    ";
    }
// запрос к таблице courses
$select_course = "SELECT * FROM `courses` WHERE
`id_personnel` = " . $_GET['id'];
$res_course = $mysqli->query($select_course);
```

```
    $rows = $res_course->fetch_all(MYSQLI_ASSOC);

    echo '<h3>Читаемые курсы</h3><hr>';

    // циклический вывод контента

    foreach ($rows as $row) {

        echo "

            <h3>{$row['name']}</h3>

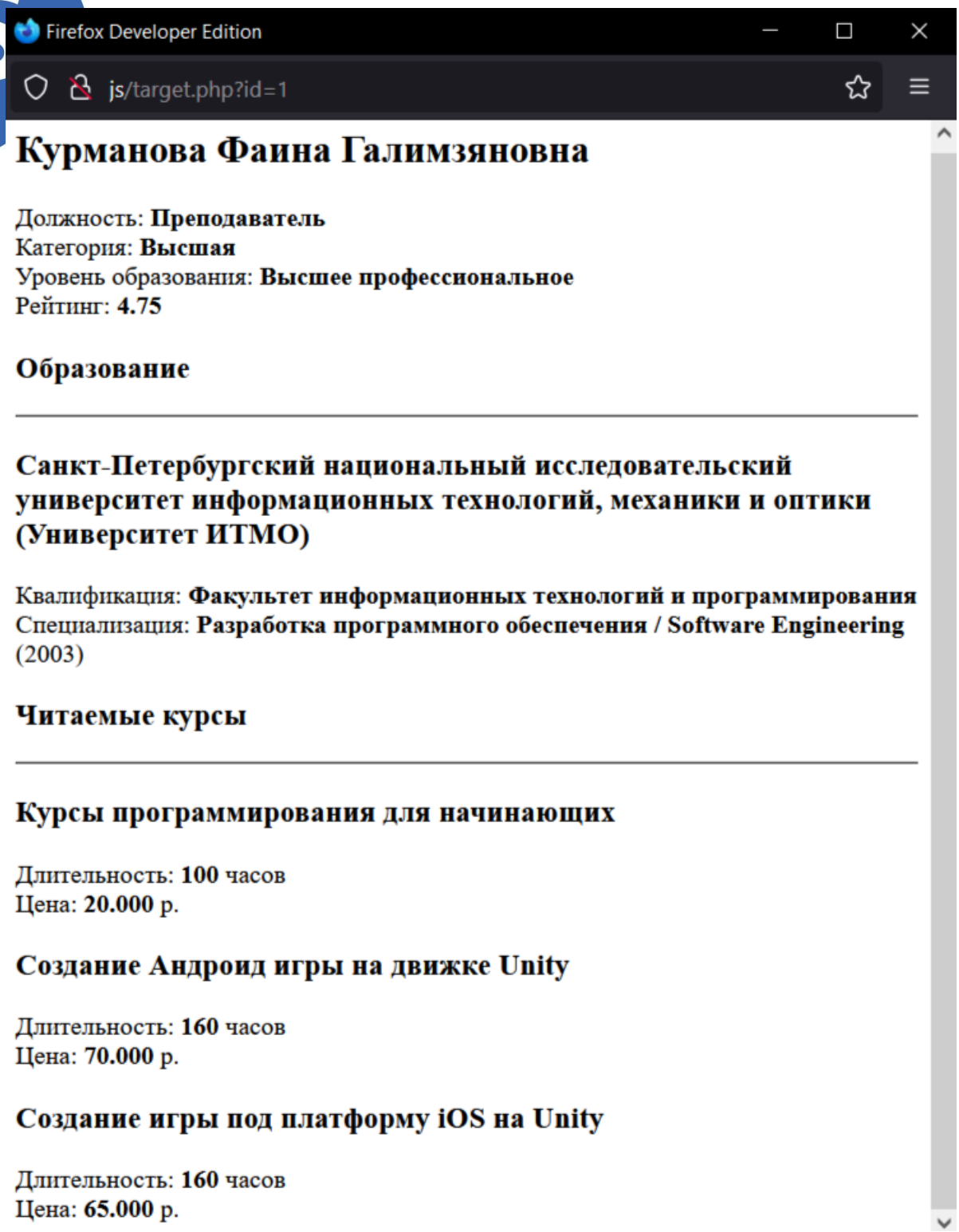
            Длительность: <b>{$row['duration']}</b> часов<br>

            Цена: <b>{$row['price']}</b> р.<br>

        ";

    }

?>
```


Firefox Developer Edition
js/target.php?id=1
Курманова Фаина Галимзяновна
Должность: Преподаватель
Категория: Высшая
Уровень образования: Высшее профессиональное
Рейтинг: 4.75
Образование
Санкт-Петербургский национальный исследовательский университет информационных технологий, механики и оптики (Университет ИТМО)
Квалификация: Факультет информационных технологий и программирования
Специализация: Разработка программного обеспечения / Software Engineering (2003)
Читаемые курсы
Курсы программирования для начинающих
Длительность: 100 часов
Цена: 20.000 р.
Создание Андроид игры на движке Unity
Длительность: 160 часов
Цена: 70.000 р.
Создание игры под платформу iOS на Unity
Длительность: 160 часов
Цена: 65.000 р.



Закрытие всплывающего окна

Всплывающее окно можно закрыть методом **close**(). Метод **close**() доступен для любого окна, но **window.close**() будет игнорироваться большинством браузеров, если window не было создано с помощью **window.open**(). Так что он сработает только для всплывающих окон.

example_16. **example_16.html**

```
<script>
    // создаем всплывающее окно
    // подгружаем в окно файл target.html
    // сохраняем ссылку
    let mywin =
    window.open('target.html','newPopUp','width=200,height=200,le
    ft=400,top=200')
    // записываем в всплывающее окно код кнопки
    mywin.document.write('<button id="btn">Кликни на
    меня</button>')
    // получаем ссылку на кнопку
    let btn = mywin.document.querySelector('#btn')
    // создаем для кнопки обработчик
    btn.addEventListener('click', () => {
        mywin.close()
    })
</script>
```

example_16. **target.html**

```
<!DOCTYPE html>
```

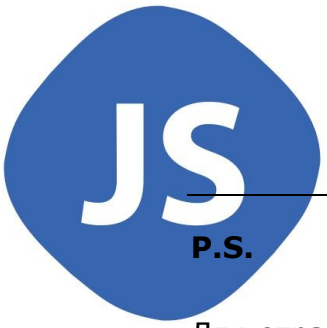
```
<html lang="ru">
    <head>
        <meta http-equiv="Content-Type" content="text/html;
        charset=utf-8">
        <title>Программирование в среде JavaScript</title>
    </head>
    <body>
    </body>
</html>
```

Пользователь может закрыть всплывающее окно в любой момент, программный код должен учитывать эту возможность. Для проверки используют метод **closed**(). Если всплывающее окно закрыто, то его свойство **closed** имеет значение **true**. Таким образом, можно легко проверить, закрыто окно или нет.

В **самостоятельных работах** к уроку выполним следующие задания:

- Создание скрипта, запускающего всплывающее окно рекламы при переходе между страницами приложения.
- Создание скрипта загрузки и автоматического закрытия всплывающего окна рекламы через 5 секунд после открытия основного файла приложения.
- Создайте скрипт вывода списка имен и краткого описания персонажей с переходом в всплывающее окно и загрузкой полной информации о выбранном персонаже.

Примечание. Если всплывающие окна созданной рекламы не похожи на те, которые вы видите в своем браузере, это просто потому, что вы не потратили время (или много времени) на их CSS-стилизацию.



P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.

К **курсу**:

- **Дополнительный практикум.**